MOVIE REVIEW SENTIMENT ANALYSIS

Development and Evaluation of a Reproducible NLP Classifier

Project Report

Course: DLBAIPNLP01 – Project: NLP

Student: Mubeen Soomro

Student number: [enter your student number]

Tutor: [enter tutor name if required]

Submission date: 31 October 2026

Contents

Abstract 3

1 Introduction 3

2 Project planning and resources 4

3 Theoretical and technical basis 6

4 Implementation and process 7

5 Evaluation and results 8

6 Discussion and reflection 9

7 Conclusion 11

References 12

Appendices 13

List of Figures and Tables

Figure 1. Confusion matrix on the official test set 9

Table 1. Project resources and constraints 5

Table 2. Test results by training-set size 8

List of Abbreviations

IMDb – Internet Movie Database; NLP – natural language processing; SVM – support vector machine; TF–IDF – term frequency–inverse document frequency; CLI – command-line interface.

List of Appendices

Appendix A. Reproduction guide 13

Abstract

This report documents the development and evaluation of a binary sentiment classifier for English-language movie reviews. The practical objective was to turn a partially implemented IMDb project into a reproducible tool that loads a public labelled dataset, preprocesses reviews, trains a supervised model, evaluates it on a separate test set, and classifies new text. The implementation uses the Stanford IMDb Large Movie Review Dataset, English stop-word filtering that preserves negation, Snowball stemming, TF–IDF unigram and bigram features, and a linear support vector classifier. Three stratified training sizes (1,000, 5,000, and 25,000 reviews) were compared against the same untouched 25,000-review test set. Accuracy increased from 83.38% to 88.93%; the full-data positive-class F1 score was 0.8890. The result meets the technical objective and demonstrates the effect of training-set scale. Reflection identifies limits from the single domain, one random seed, and a classical bag-of-words representation, and proposes controlled follow-up experiments.

1 Introduction

1.1 Problem and context

Online reviews give audiences a way to express opinions about films, but manually reading thousands of comments is costly. A sentiment analysis system can help summarize a large review collection by assigning each text to a positive or negative class. This project addresses the course task of building such a system and evaluating its performance on held-out data. The chosen setting is the Stanford IMDb benchmark, which contains balanced positive and negative movie reviews (Maas et al., 2011). It is a suitable first test because its labels and official train/test split are already provided, allowing the work to focus on a controlled NLP workflow rather than an unverified web-scraping process.

1.2 Objective and success criteria

The main objective was to deliver a reproducible end-to-end classifier and document the decisions needed to reproduce it. Success was defined by five observable criteria: (1) load all 25,000 training and 25,000 test reviews with balanced labels; (2) implement text cleaning, stop-word handling, stemming, and numerical text encoding; (3) train a supervised model without using test labels during fitting; (4) report accuracy, precision, recall, and F1 on the official test

split; and (5) expose a simple command-line interface for classifying an unseen review. A further criterion was to compare progressively larger training subsets on the same test set, as requested in the task brief.

1.3 Approach and report structure

The work began by inspecting the existing repository, dataset layout, starter data loader, and exploratory notebook. The notebook contained stale output showing an empty training frame and a missing-label error. The implementation was then repaired and separated into modules for data handling, model construction, training/evaluation, and prediction. The model was run on three reproducible training sizes, and the output artifacts were checked. Section 2 records planning and resources; Section 3 explains the theoretical basis; Section 4 describes the implementation process; Section 5 presents measured results; Section 6 evaluates the result and reflects on limitations; Section 7 concludes with future work.

2 Project planning and resources

2.1 Project phases and decision rationale

The project was organized into four phases. First, repository and data inspection established what was available and exposed the stale notebook result. Second, the data workflow was made reliable: expected folder structure, class counts, and dataset archive checksum are checked so an incomplete dataset cannot silently pass as valid. Third, the modelling workflow was implemented as a scikit-learn pipeline. Keeping vectorization and classification together ensures that vocabulary and inverse document frequencies are fitted only on each training subset. Fourth, evaluation and documentation were produced, including the training-size comparison, model file, class report, confusion matrix, and reproducibility instructions. The phases were chosen to address data reliability before model complexity; a classifier score is not meaningful if the sample or evaluation split is wrong.

The scope was kept to one classical supervised approach rather than several unrelated models. A linear SVM is appropriate for sparse, high-dimensional text features and provides a strong, interpretable baseline in terms of a separating margin (Joachims, 1998). TF–IDF downweights terms that occur broadly and gives more influence to terms that are distinctive to documents; bigrams retain some local word-order information. Snowball stemming reduces inflectional

variation, while stop-word removal limits common function-word features. Negation terms were explicitly retained because removing them can reverse a sentence's sentiment. These choices balance performance, runtime, and ease of reproduction.

The task brief also cites work comparing classical and deep-learning approaches across text domains (Paramesha et al., 2023). That comparison supports treating model choice as a trade-off rather than assuming that a neural model is automatically preferable. This project selected a linear SVM because it is a well-established sparse-text baseline that can be trained and evaluated with the available resources. A deep model was left as future comparative work; no claim is made here that the SVM is more accurate than deep-learning alternatives.

2.2 Resources and constraints

Resource	Use	Constraint / control
IMDb Large Movie Review Dataset	50,000 labelled reviews; official split	Local copy; archive checksum and split counts validated
Python 3.13 virtual environment	Execution and dependency isolation	Existing environment; versions recorded by installed packages
pandas / scikit-learn / NLTK	Data frames, TF–IDF, LinearSVC, stemming	Classical baseline; no GPU required
Matplotlib / joblib	Confusion matrix and model persistence	Generated artifacts stored under results/
GitHub repository	Version control and code access	Changes remain local until explicitly pushed

No paid services or additional hardware were required. The dataset is public and the project does not collect personal information from participants. A practical constraint was that the dataset consists of tens of thousands of small files; the loader uses bounded parallel reads to reduce the time spent on file I/O. The experiment is repeatable but fit times depend on machine and storage performance.

2.3 Risks and mitigation

The main risks were incomplete data, leakage between training and test, preprocessing errors, and irreproducible evaluation. Incomplete data is addressed by checking all four class folders and exact review counts. Leakage is limited by fitting a fresh pipeline for each training size and keeping the official test split separate. Preprocessing retains negation and handles text decoding errors explicitly. Reproducibility is supported by fixed random seeds, recorded metrics, an explicit CLI, and a serialized model. A remaining methodological risk is that a single split and seed do not quantify statistical uncertainty; this is discussed in Section 6.

3 Theoretical and technical basis

3.1 Text classification and supervised learning

In supervised text classification, each document is paired with a target label. A learning algorithm estimates a decision function from labelled examples and then applies that function to new documents. In this project the labels are binary: 0 for negative and 1 for positive. The system's generalization is assessed on examples excluded from fitting. This separation matters because evaluating on training data would measure memorization as well as learning and would overstate expected performance on unseen reviews (Jurafsky & Martin, 2013).

3.2 Feature representation and margin-based classification

Raw review strings cannot be passed directly to a conventional linear classifier, so the vectorizer maps each review into a sparse numerical feature vector. TF–IDF represents terms using their within-document frequency and inverse corpus frequency. Unigrams capture individual words; bigrams capture adjacent pairs such as "not good." Sublinear term frequency dampens the effect of repeated occurrences. The linear SVM finds a separating hyperplane with a large margin between positive and negative training examples. Sparse linear models are useful for text because many token features occur in only a small fraction of documents and the feature dimension can be large relative to each document (Joachims, 1998).

3.3 Preprocessing as a modelling decision

Preprocessing changes the information available to a classifier. Lowercasing merges case variants, tokenization identifies word-like units, and stemming maps related inflections toward a shared stem. Stop-word filtering can reduce common low-information terms, but blanket

removal is risky: “not” is frequent and short, yet semantically decisive. The implementation therefore excludes a short set of negation cues from the stop-word list. This is an example of transferring linguistic knowledge into a practical pipeline rather than applying preprocessing mechanically. Bigrams partly preserve local ordering, but the representation still does not understand syntax, sarcasm, or long-distance context.

4 Implementation and process

4.1 Data loading and validation

The project uses the official train/test partition distributed with the IMDb dataset. Each split contains 12,500 positive and 12,500 negative reviews. The loader builds a row for each review, retaining review ID, text, binary sentiment, rating, and an associated movie identifier where available. It checks the dataset archive’s MD5 value when a download is needed and validates the extracted class folders before returning data. If a split is incomplete, it raises an error instead of returning an apparently valid but partial frame. This check directly addresses the initial project’s risk of confusing stale notebook output with current data status.

4.2 Feature pipeline and model

The model pipeline consists of a custom tokenizer/stemmer, TF–IDF vectorization over unigrams and bigrams, and LinearSVC with C=1.0 and a fixed classifier seed. English stop words are removed before stemming, with negation terms preserved. The vectorizer caps vocabulary size at 200,000 and ignores terms occurring in fewer than two training documents. The pipeline ensures the vectorizer’s vocabulary and inverse-document frequencies are learned from the training subset, not the test corpus. The full training split is used to fit the final serialized model in `results/sentiment_model.joblib`.

4.3 Progressive-size experiment

To test the effect of sample size, three stratified subsets of the official training set were selected: 1,000, 5,000, and 25,000. The two smaller samples use a fixed random seed of 42 and preserve the class balance. Each model was evaluated on all 25,000 official test reviews. The test set was not used for selecting features, thresholds, or hyperparameters. The approach therefore compares learning curves under a controlled test condition, although it represents one sample draw for each size rather than repeated sampling.

4.4 Difficulties and resolution

The initial notebook contained stored output from an earlier unsuccessful run: the displayed training frame had zero rows, and the next cell failed on the missing `label` column. This output was not accepted as evidence of the present dataset state. Inspection confirmed that the local dataset had the expected class folders and review counts. The loader was hardened to validate completeness, and the notebook was replaced with a clean exploration workflow that loads current data and reports split counts, duplicates, and review lengths. A second implementation issue was that a full-size training request cannot be passed to a splitting helper as a training portion while also requiring a remainder; the experiment now uses the complete training frame directly for that final scenario.

4.5 Product and typical use

The completed product is a reusable classifier rather than a one-off notebook. A user trains the system with `python -m src.train`, which creates CSV and JSON metrics, a fitted model, a class-wise report, and a confusion-matrix image. A new review can be classified with `python -m src.predict "review text"`; the command returns `positive` or `negative`. The modular code supports inspection and extension, while the notebook remains an optional exploration surface. The repository contains setup instructions, the source code, report, and generated evaluation artifacts. The dataset itself remains excluded from version control.

5 Evaluation and results

Accuracy measures the fraction of correct predictions. Precision measures the fraction of positive predictions that are truly positive, recall measures the fraction of true positive reviews found, and F1 is their harmonic mean. The dataset is balanced, but reporting all four measures makes it possible to see whether the model favors one class. Fit time is included as a practical resource measure and excludes dataset loading and prediction.

Train n	Test n	Accuracy	Precision+	Recall+	F1+	Fit (s)
1,000	25,000	83.38%	83.04%	83.90%	83.47%	0.83
5,000	25,000	86.76%	86.23%	87.50%	86.86%	5.24
25,000	25,000	88.93%	89.13%	88.67%	88.90%	20.39

The full-data model correctly classified 22,232 of 25,000 test reviews. For the negative class, precision was 0.8873, recall 0.8918, and F1 0.8896. For the positive class, precision was 0.8913, recall 0.8867, and F1 0.8890. The close class-wise scores indicate balanced performance on this balanced benchmark. The training-size comparison shows a 5.54 percentage-point increase in accuracy between the 1,000- and 25,000-review conditions. Most of the observed gain occurs by 5,000 examples, with a further 2.16 percentage-point increase from 5,000 to 25,000.

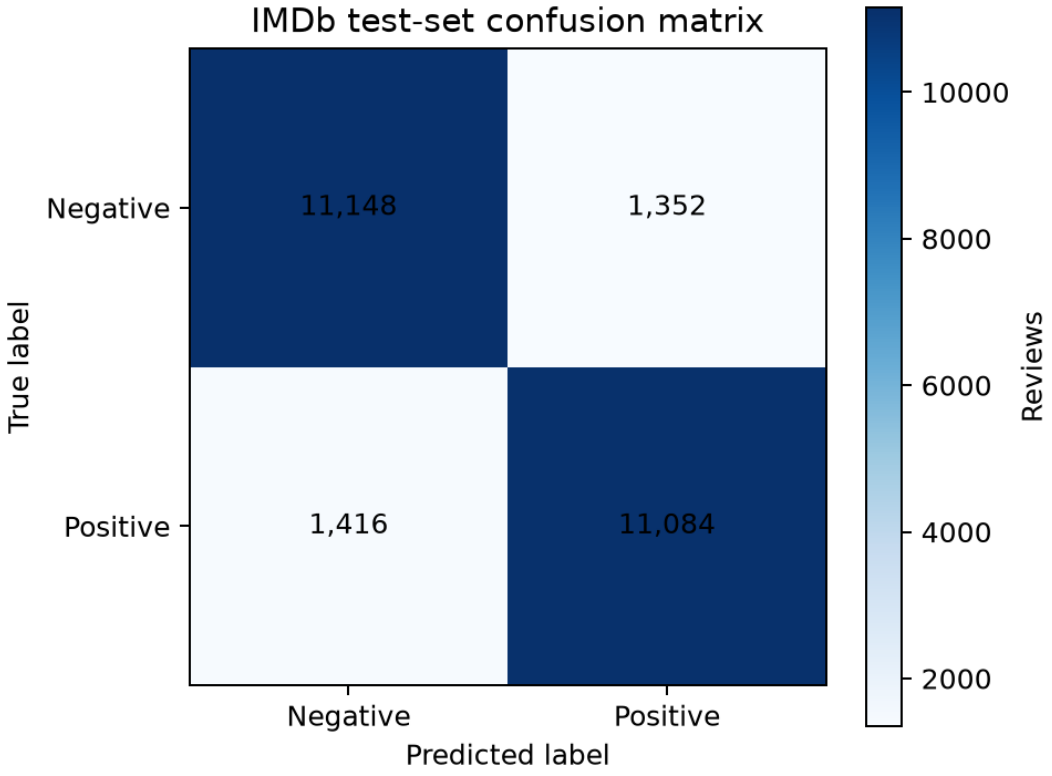


Figure 1. Confusion matrix on the 25,000-review official test set. Rows show the true class and columns the predicted class.

The matrix contains 11,148 true negatives, 1,352 negative reviews predicted positive, 1,416 positive reviews predicted negative, and 11,084 true positives. False positives and false negatives are similar in number, consistent with the class-wise metrics. The result is useful as a baseline, but an accuracy below 90% leaves a meaningful error rate of 11.07%; the system should not be treated as a perfect substitute for human interpretation of reviews.

6 Discussion and reflection

6.1 Goal achievement and transfer to practice

All technical success criteria were met: the full balanced dataset loads, preprocessing and numerical encoding are implemented, a supervised classifier is trained, a held-out test evaluation is reported, and a command-line prediction interface is available. The project also follows the requested progression from a small sample to larger training data. The practical conclusion is that a compact classical pipeline can produce useful sentiment summaries with modest computational resources. The code can support batch analysis, but its output should be interpreted as an estimate of polarity rather than as a complete account of audience opinion.

Theoretical ideas guided practical decisions. The supervised-learning framework motivated a strict train/test separation. The margin-based SVM was selected for sparse high-dimensional text, and TF–IDF provides a conventional weighting scheme for lexical features. Linguistic awareness motivated preserving negation. The measured scaling curve shows why dataset size is a resource decision: additional labelled data improved accuracy, but the marginal gain from 5,000 to 25,000 examples was smaller than the gain from 1,000 to 5,000. This finding can guide later work toward targeted error analysis rather than assuming that simply adding data will solve every failure mode.

6.2 Quality, limitations, and validity

The strongest quality of the evaluation is the use of the official held-out test split and repeated use of that same split across the three training sizes. The benchmark is balanced, all primary classification measures are reported, and the confusion matrix exposes error counts. The limitations are also material. First, the result is specific to English movie reviews and may not transfer to product reviews, social media, or other domains. Second, a single random seed provides no estimate of variation across training samples. Third, no hyperparameter search or alternative model comparison was performed, so the score is not claimed to be optimal. Fourth, token-level features do not represent sarcasm, discourse, or contextual meaning robustly. Finally, the training size changes vocabulary and IDF estimates as well as the number of labelled examples, which is part of the practical learning-curve effect but does not isolate every mechanism.

6.3 Improvements and sustainable integration

A next iteration should first conduct error analysis on false positives and false negatives, grouping examples by phenomena such as negation scope, sarcasm, and mixed sentiment. A second experiment should repeat each training size with several fixed seeds and report mean and dispersion, while preserving the official test set for final evaluation. Hyperparameters could be selected by cross-validation on training data only. A comparison with a Naive Bayes baseline and a pretrained transformer would clarify the trade-off between accuracy, runtime, and explainability. For sustainable use, the project should pin dependency versions, add automated checks for split integrity and model loading, record a model version with its training configuration, and define a human review process for decisions where sentiment errors could have consequences. These improvements build on the existing modular product rather than replacing it without evidence.

7 Conclusion

This project transformed an incomplete starter repository into a reproducible IMDb sentiment analysis system. The final LinearSVC pipeline achieved 88.93% accuracy on the official test set and showed steadily improving results as training data increased from 1,000 to 25,000 reviews. The project demonstrates a practical transfer of supervised learning, TF–IDF representation, stemming, and margin-based classification to a real NLP task. The main professional lesson is that a credible model result depends on the integrity of the data and evaluation process as much as on the classifier itself. The delivered system is a documented baseline suitable for extension, not a claim of perfect sentiment understanding.

The work also provides a practical foundation for further study in NLP. Future experiments can build on the same validated data split and compare lexical models with neural representations, while preserving a final test set that remains untouched during model selection. The observed gains from larger samples suggest that data quantity helps, but the remaining error rate shows that quantity alone is insufficient. A careful review of false classifications can identify which language phenomena deserve attention before computationally expensive methods are introduced.

For professional practice, the project reinforces three habits: validate data before trusting model output, connect preprocessing choices to the meaning of language, and report limitations

alongside performance. If the classifier were integrated into an organization's review monitoring process, it should be used to summarize trends rather than make individual decisions. Human review, monitoring for domain changes, versioned model artifacts, and periodic re-evaluation would be needed for sustainable use. These controls link technical performance to responsible implementation.

The final result therefore meets the stated objectives and creates a clear path for continuous improvement. Repeated-seed evaluation, error analysis, and cross-domain testing are the next priorities. A more complex model should be adopted only if those experiments show that its added accuracy justifies its cost and reduced interpretability. This sequence makes future development evidence-led and supports responsible use of the system.

References

IU International University of Applied Sciences. (n.d.). Project report: Guidelines for the creation of a project report. Examination Office.

Jurafsky, D., & Martin, J. H. (2013). Speech and language processing: An introduction to natural language processing, computational linguistics, and speech recognition (2nd ed.). Pearson Prentice Hall.

Joachims, T. (1998). Text categorization with Support Vector Machines: Learning with many relevant features. In C. Nédellec & C. Rouveirol (Eds.), Machine Learning: ECML-98 (pp. 137–142). Springer. https://doi.org/10.1007/BFb0026683

Maas, A. L., Daly, R. E., Pham, P. T., Huang, D., Ng, A. Y., & Potts, C. (2011). Learning word vectors for sentiment analysis. Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies, 142–150. https://ai.stanford.edu/~amaas/data/sentiment/

Paramesha, K., Gururaj, H. L., Nayyar, A., & Ravishankar, K. C. (2023). Sentiment analysis on cross-domain textual data using classical and deep learning approaches. Multimedia Tools and Applications, 1–24.

NLTK Project. (n.d.). Snowball stemmer. https://www.nltk.org/api/nltk.stem.snowball.html

scikit-learn developers. (n.d.). TfidfVectorizer and LinearSVC documentation. https://scikit-learn.org/stable/

Appendix A Reproduction guide

From the `imdb-sentiments` project directory, create and activate a virtual environment, install the packages in `requirements.txt`, and run `python -m src.train`. The first run downloads and validates the public dataset if it is not already present. Default training sizes are 1,000, 5,000, and 25,000. Metrics and the full model are written under `results/`. Run `python -m src.predict "A warm and satisfying film."` to classify a new review. The upstream code repository is https://github.com/MubeenAhmed23/imdb-sentiments. The source tree and locally generated report/results should be committed and pushed before the repository is used as a submission reference.

The report is based on measured run outputs in `results/metrics.csv`, `results/classification_report.txt`, and `results/figures/confusion_matrix.png`. Fit times vary by environment; predictive metrics should remain reproducible when package versions and random seeds are held constant.